IITGN Connect

A College Social Media Platform

CS 432: Databases
Assignment 2: Module A & Module B

Semester II, 2025–2026

Team Members	Module	Role
Laksh Jain	A	B+ Tree DBMS Engine
Rudra Pratap Singh	A	B+ Tree DBMS Engine
Parthiv Patel	B	Backend & Database Design
Shriniket Behera	B	RBAC & Optimization
Ridham Patel	B	Frontend, UI/UX & Security

GitHub: https://github.com/23110281/db_track1_assignment2

Indian Institute of Technology, Gandhinagar

Contents

Module A: Lightweight DBMS with B+ Tree Index 3

1 Introduction 3

2 Implementation Details 3
- 2.1 File Structure . . . 3
- 2.2 B+ Tree (`database/bplustree.py`) . . . 3
  - 2.2.1 Node Structure . . . 4
  - 2.2.2 Insertion . . . 4
  - 2.2.3 Deletion . . . 5
  - 2.2.4 Range Queries . . . 5
- 2.3 BruteForceDB (`database/bruteforce.py`) . . . 6
- 2.4 Table Abstraction (`database/table.py`) . . . 6
- 2.5 Database Manager (`database/db_manager.py`) . . . 6
- 2.6 Flask REST API (`app.py` & `routes.py`) . . . 7

3 Visualisation 7
- 3.1 Initial Tree (after insertion) . . . 8
- 3.2 Tree After Deletions . . . 8
- 3.3 Tree After New Insertions . . . 8

4 Performance Benchmarking 9
- 4.1 Methodology . . . 9
- 4.2 Results Table (all times in seconds) . . . 9
- 4.3 Results Summary . . . 10
- 4.4 Insertion Time . . . 11
- 4.5 Search Time . . . 11
- 4.6 Range Query Time . . . 12
- 4.7 Deletion Time . . . 12
- 4.8 Memory Usage (Peak at Insertion) . . . 13

5 Comprehensive Tests 13

6 Module A: Conclusion 14

Module B: Local API Development, RBAC & Database Optimisation 15

7 Local Database & Secure API Implementation 15
- 7.1 Tech Stack . . . 15
- 7.2 Database Schema (22 Tables) . . . 15
  - 7.2.1 Data Integrity Strategy . . . 15
  - 7.2.2 Key Table Definitions . . . 16
- 7.3 CRUD API Endpoints . . . 17

7.4 Session Validation . . . 18
7.5 Member Portfolio (Profile) . . . 19

8 Security & Role-Based Access Control (RBAC) 20
8.1 Role-Based Access Control . . . 20
8.1.1 Implementation Details . . . 20
8.2 Role-Based UI Visibility . . . 21
8.3 Proper Deletion Integrity . . . 21
8.4 Unauthorized Access Logging . . . 21
8.4.1 Layer 1: Unauthorized API Request Detection . . . 21
8.4.2 Layer 2: Direct DB Modification Detection (63 MySQL Triggers) 22

9 Database Optimization 24
9.1 Indexing Strategy . . . 24
9.1.1 Index Design Principles . . . 24
9.1.2 Complete Index Listing . . . 24
9.2 Query Profiling . . . 24
9.2.1 Benchmark Methodology . . . 24
9.2.2 Why IGNORE INDEX Instead of DROP/CREATE? . . . 25

10 Optimization Report 26
10.1 Execution Time Results . . . 26
10.2 Scan Type Analysis (EXPLAIN ANALYZE) . . . 26
10.3 Benchmarking Charts . . . 27
10.4 EXPLAIN Plan Analysis . . . 29
10.4.1 Detailed EXPLAIN Comparison: Global Post Feed . . . 29
10.4.2 Detailed EXPLAIN Comparison: Group Posts . . . 30

11 Video Walkthrough 31
11.1 UI & API Functionality . . . 31
11.2 RBAC Constraints . . . 31
11.3 Security Logging . . . 31

Module A: Lightweight DBMS with B+ Tree Index

Introduction

Efficient data retrieval is a core challenge in database systems. The **B+ Tree** is a self-balancing, ordered tree structure that excels at both point lookups and range scans because all records live in the leaf level, which is linked as a doubly-chained list.

This module implements a B+ Tree indexing engine from scratch in Python and compares it with a naïve **BruteForceDB** (a plain Python list) to quantify:

Operation	B+ Tree Complexity	BruteForce Complexity
Search	$O(\log n)$	$O(n)$
Insert	$O(\log n)$	$O(1)$ amortised
Delete	$O(\log n)$	$O(n)$
Range Query	$O(\log n + k)$	$O(n)$

Table 1: Asymptotic complexity comparison: B+ Tree vs BruteForceDB.

Implementation Details

File Structure

```
ModuleA/
  database/
    __init__.py
    bplustree.py       # B+ Tree implementation
    bruteforce.py       # BruteForceDB baseline
    table.py            # Table abstraction over B+ Tree
    db_manager.py       # Database manager (multiple DBs/tables)
  app.py                # Flask application factory
  routes.py             # REST API endpoints
  report.ipynb          # Jupyter notebook report & benchmarks
  requirements.txt
```

B+ Tree (`database/bplustree.py`)

The `BPlusTree` class uses two node types:

- **Internal nodes**: store separator keys and child pointers.
- **Leaf nodes**: store key-value pairs and are linked via a `next` pointer, enabling efficient range scans.

Key operations implemented:

Method	Description
`insert(key, value)`	Inserts a key-value pair; auto-splits full nodes up the tree.
`delete(key)`	Removes a key with proper merge/borrow rebalancing.
`search(key)`	Traverses from root to leaf; returns associated value or `None`.
`range_query(start, end)`	Walks the leaf chain collecting all keys in [start, end].
`update(key, new_value)`	Updates the value associated with an existing key.
`get_all()`	In-order traversal via leaf chain; returns all key-value pairs.
`visualize_tree()`	Generates a Graphviz `Digraph` of the tree structure.

Table 2: B+ Tree methods summary.

2.2.1 Node Structure

```python
class BPlusTreeNode:
    def __init__(self, order, is_leaf=False):
        self.order = order
        self.is_leaf = is_leaf
        self.keys = []
        self.children = []    # child pointers (internal nodes)
        self.values = []      # associated values (leaf nodes)
        self.next = None      # linked-list pointer (leaf nodes)

    def is_full(self):
        return len(self.keys) == self.order - 1
```

Listing 1: BPlusTreeNode class

2.2.2 Insertion

Insertion uses the standard B+ Tree algorithm: traverse to the appropriate leaf, insert the key in sorted order, and split upward if the node is full. The root is split by creating a new root when necessary.

```python
def insert(self, key, value):
    root = self.root
    if root.is_full():
        new_root = BPlusTreeNode(self.order, is_leaf=False)
        self.root = new_root
        new_root.children.append(root)
        self._split_child(new_root, 0)
        self._insert_non_full(new_root, key, value)
    else:
        self._insert_non_full(root, key, value)

def _split_child(self, parent, index):
    node = parent.children[index]
    new_node = BPlusTreeNode(self.order, is_leaf=node.is_leaf)
    if node.is_leaf:
        mid = len(node.keys) // 2
        new_node.keys = node.keys[mid:]
        new_node.values = node.values[mid:]
```

```
        node.keys = node.keys[:mid]
        node.values = node.values[:mid]
        new_node.next = node.next    # preserve leaf chain
        node.next = new_node
        parent.keys.insert(index, new_node.keys[0])  # copy up
    else:
        mid = len(node.keys) // 2
        up_key = node.keys[mid]      # promote middle key
        new_node.keys = node.keys[mid+1:]
        new_node.children = node.children[mid+1:]
        node.keys = node.keys[:mid]
        node.children = node.children[:mid+1]
        parent.keys.insert(index, up_key)
    parent.children.insert(index + 1, new_node)
```

Listing 2: Insert and split logic (excerpt)

2.2.3 Deletion

Deletion traverses to the target leaf, removes the key, and triggers rebalancing when a node falls below the minimum key count. Rebalancing uses:

- **Borrow from left sibling**: rotate the largest key from the left sibling through the parent.
- **Borrow from right sibling**: rotate the smallest key from the right sibling through the parent.
- **Merge**: combine the underflowing node with a sibling and pull down the separator from the parent.

If the root becomes empty after a merge, the tree's height shrinks by replacing the root with its sole child.

2.2.4 Range Queries

Range queries exploit the leaf-level linked list. The algorithm locates the starting leaf via a root-to-leaf traversal ($O(\log n)$), then walks the `next` pointers collecting all keys in the range ($O(k)$ for k matching keys).

```
def range_query(self, start_key, end_key):
    node = self.root
    while not node.is_leaf:
        i = bisect.bisect_right(node.keys, start_key)
        node = node.children[i]
    results = []
    while node:
        for k, v in zip(node.keys, node.values):
            if start_key <= k <= end_key:
                results.append((k, v))
            elif k > end_key:
                return results
        node = node.next
    return results
```

Listing 3: Range query using leaf chain

BruteForceDB (database/bruteforce.py)

The `BruteForceDB` class serves as a baseline for performance comparison. It stores keys in an unsorted Python list and performs all operations via linear iteration.

```python
class BruteForceDB:
    def __init__(self):
        self.data = []

    def insert(self, key):
        self.data.append(key)

    def search(self, key):
        return key in self.data

    def delete(self, key):
        if key in self.data:
            self.data.remove(key)

    def range_query(self, start, end):
        return [k for k in self.data if start <= k <= end]
```

Listing 4: BruteForceDB class

Table Abstraction (database/table.py)

The `Table` class wraps the B+ Tree to provide a record-based database abstraction with schema validation. It accepts dictionary records, uses a designated `search_key` field as the B+ Tree key, and delegates all indexing operations to the underlying tree.

```python
class Table:
    def __init__(self, name, schema, order, search_key):
        self.name = name
        self.schema = schema
        self.search_key = search_key
        self.data = BPlusTree(order=order)

    def insert(self, record):
        self.validate_record(record)
        key = record[self.search_key]
        self.data.insert(key, record)

    def range_query(self, field, start_value, end_value):
        if field == self.search_key:
            results = self.data.range_query(start_value, end_value)
            return [v for k, v in results]
        else:  # fallback: scan all records
            return [rec for k, rec in self.data.get_all()
                    if start_value <= rec.get(field, None) <= end_value
                    ]
```

Listing 5: Table class (excerpt)

Database Manager (database/db_manager.py)

The `DatabaseManager` class manages multiple named databases, each containing multiple named tables. It provides CRUD operations at both the database and table levels.

```python
class DatabaseManager:
    def __init__(self):
        self.databases = {}

    def create_database(self, db_name):
        if db_name not in self.databases:
            self.databases[db_name] = {}
            return True
        return False

    def create_table(self, db_name, table_name, schema, order,
        search_key):
        if db_name not in self.databases:
            return False
        if table_name not in self.databases[db_name]:
            self.databases[db_name][table_name] = Table(
                table_name, schema, order, search_key)
            return True
        return False
```

Listing 6: DatabaseManager class

Flask REST API (`app.py` & `routes.py`)

A lightweight Flask API exposes the DBMS engine over HTTP, enabling programmatic interaction:

Method	Endpoint	Description
GET	`/databases`	List all databases
POST	`/databases`	Create a database
DELETE	`/databases/:db`	Delete a database
GET	`/databases/:db/tables`	List tables
POST	`/databases/:db/tables`	Create a table (schema, order, key)
DELETE	`/databases/:db/tables/:tbl`	Delete a table
POST	`/databases/:db/tables/:tbl/records`	Insert a record
GET	`/databases/:db/tables/:tbl/records`	Get all records
GET	`/databases/:db/tables/:tbl/records/:id`	Get record by key
PUT	`/databases/:db/tables/:tbl/records/:id`	Update record by key
DELETE	`/databases/:db/tables/:tbl/records/:id`	Delete record by key
POST	`/databases/:db/tables/:tbl/search`	Search by constraints
GET	`/databases/:db/tables/:tbl/range`	Range query (field, start, end)
GET	`/databases/:db/tables/:tbl/visualize`	Render B+ Tree as SVG

Table 3: Module A REST API endpoints.

Visualisation

The `visualize_tree()` method generates a Graphviz `Digraph` with:

- **Internal nodes** rendered in blue with bullet (•) pointers between separator keys.
- **Leaf nodes** rendered in green showing key/value pairs in a two-row HTML table.
- **Parent to child edges**: solid arrows from pointer slots to child nodes.

- **Leaf chain links**: dashed blue arrows between consecutive leaf nodes (the `next` pointers that enable range scans).

Initial Tree (after insertion)

A small, hand-crafted set of keys is inserted so the resulting tree is easy to inspect visually.

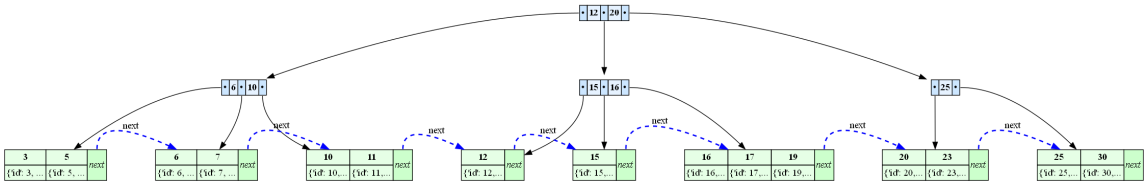

Figure 1: B+ Tree after initial insertions. Solid arrows = parent to child links; dashed arrows = leaf-chain links.

Observations:

- The root contains the two separator keys that split the three leaf nodes.
- Leaf nodes are linked left-to-right (dashed edges), forming a sorted sequential chain used by range queries.

Tree After Deletions

Several keys are deleted to demonstrate node merging and borrowing.

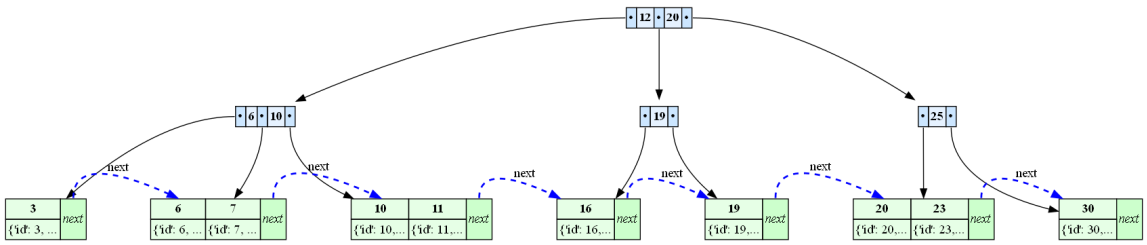

Figure 2: B+ Tree after deleting keys 5, 12, 17, 25, 15. Rebalancing (borrow/merge) maintains tree invariants.

Observations:

- Removing keys triggers rebalancing (borrow-from-sibling or merge).
- The tree height may shrink when the root empties after a merge.

Tree After New Insertions

New keys (50, 55, 60, 65, 70) are inserted to demonstrate tree growth and node splitting.

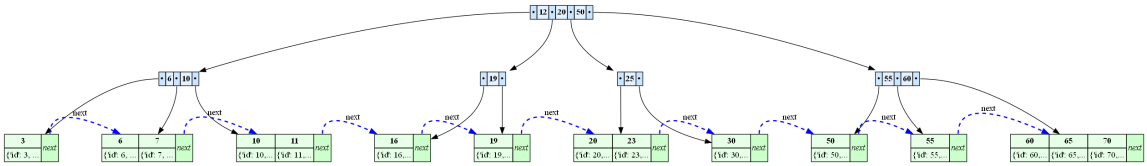

Figure 3: B+ Tree after inserting additional keys. Node splitting produces new leaf and internal nodes.

Performance Benchmarking

Methodology

Both structures are benchmarked across six dataset sizes: $N \in \{100, 500, 1000, 2000, 4000, 8000\}$ records. For each size:

1. **Insert** N randomly sampled unique keys from $[1, 100\,000]$.
2. **Search** for a random 10% subset of those keys.
3. **Range query** over 10 random narrow intervals (width 100–1 000).
4. **Delete** the same 10% subset used in step 2.

Each operation is timed with `time.time()` and peak memory is measured via `tracemalloc`. The B+ Tree is initialised with `order=50` (mimicking realistic page sizes) to amortise splitting costs.

Results Table (all times in seconds)

	Insert		Search		Range		Delete		Mem (KB)	
N	**BPT**	**BF**	**BPT**	**BF**	**BPT**	**BF**	**BPT**	**BF**	**BPT**	**BF**
100	0.0020	0.0000	0.0000	0.0000	0.0010	0.0000	0.0000	0.0000	3.36	1.88
500	0.0020	0.0010	0.0000	0.0000	0.0010	0.0000	0.0010	0.0000	15.14	8.31
1000	0.0053	0.0010	0.0000	0.0010	0.0000	0.0010	0.0000	0.0021	33.34	17.38
2000	0.0060	0.0020	0.0000	0.0046	0.0000	0.0010	0.0010	0.0060	67.01	31.69
4000	0.0117	0.0010	0.0010	0.0080	0.0000	0.0020	0.0010	0.0205	140.97	64.63
8000	0.0280	0.0020	0.0010	0.0306	0.0010	0.0046	0.0021	0.0683	283.70	131.38

Table 4: B+ Tree vs BruteForceDB raw timings. BPT = B+ Tree, BF = BruteForceDB.

Results Summary

Operation	B+ Tree	BruteForceDB
Insert	Slower at small N due to node splitting overhead; grows $O(N \log N)$ total.	Faster at small N (`list.append` is $O(1)$ amortised in C).
Search	Near-flat $O(\log N)$, many times faster at 8 000 records.	Linear $O(N)$ growth.
Range Query	$O(\log N + k)$ via leaf chain; skips irrelevant records.	Full $O(N)$ scan every time.
Deletion	$O(\log N)$ including rebalancing.	$O(N)$ locate + shift.
Memory	Higher per-element due to node objects and pointers.	Lower (raw Python list).

Table 5: Qualitative comparison of B+ Tree vs BruteForceDB.

Key observations:

- **Insert**: BruteForceDB's $O(1)$ append beats B+ Tree at small N. B+ Tree incurs overhead from node splitting and pointer rewiring, but remains tractable for large datasets.
- **Search**: B+ Tree's $O(\log N)$ traversal dominates at every scale. At 8 000 records it is already many times faster than BruteForceDB's linear scan.
- **Range Query**: With narrow windows (width 100–1 000), the B+ Tree locates the start in $O(\log N)$ then walks only matching leaves. BruteForceDB must scan all N elements.
- **Deletion**: BruteForceDB must first find the element ($O(N)$) and then shift subsequent elements. B+ Tree deletion, even with borrowing and merging, remains $O(\log N)$.
- **Memory**: B+ Tree uses more memory per element due to node objects and pointers. BruteForceDB's raw list is more compact. In practice, this trade-off is worthwhile because indexed lookups eliminate full scans.

Insertion Time

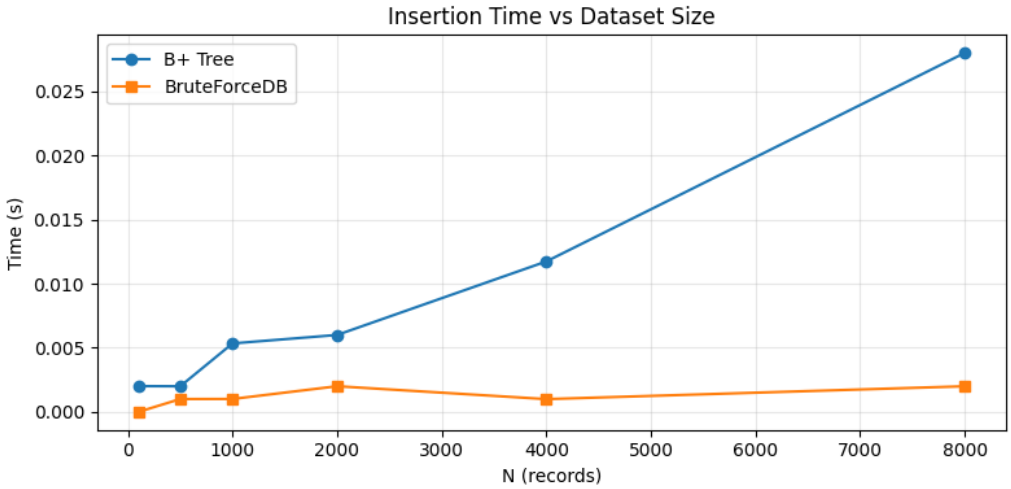

Figure 4: Insertion time: B+ Tree vs BruteForceDB.

Analysis: BruteForceDB's $O(1)$ append keeps it faster for inserts at smaller N. B+ Tree incurs overhead from node splitting and pointer rewiring; however, its growth remains $O(N \log N)$ in total, still tractable for large datasets.

Search Time

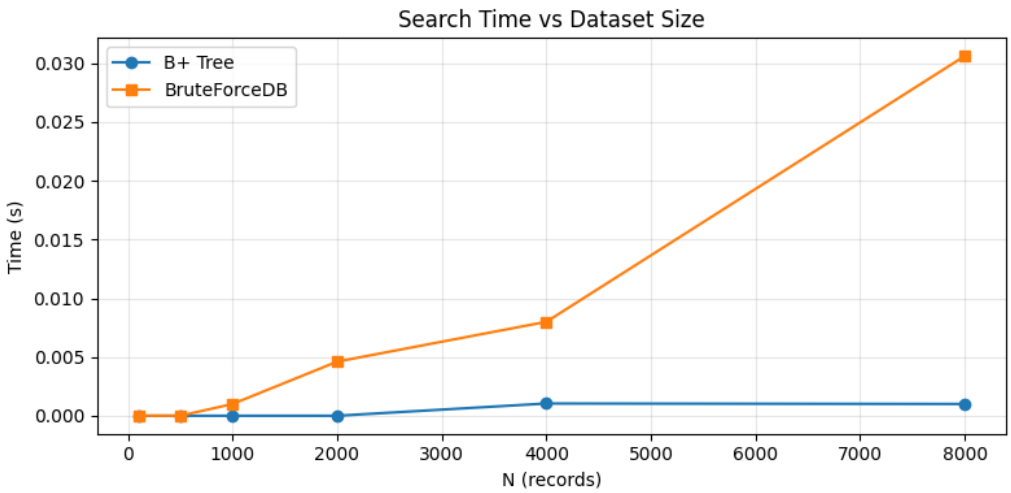

Figure 5: Search time: B+ Tree vs BruteForceDB.

Analysis: BruteForceDB's search time grows linearly ($O(N)$), while the B+ Tree search remains near-flat ($O(\log N)$). At 8 000 records the B+ Tree is already many times faster.

Range Query Time

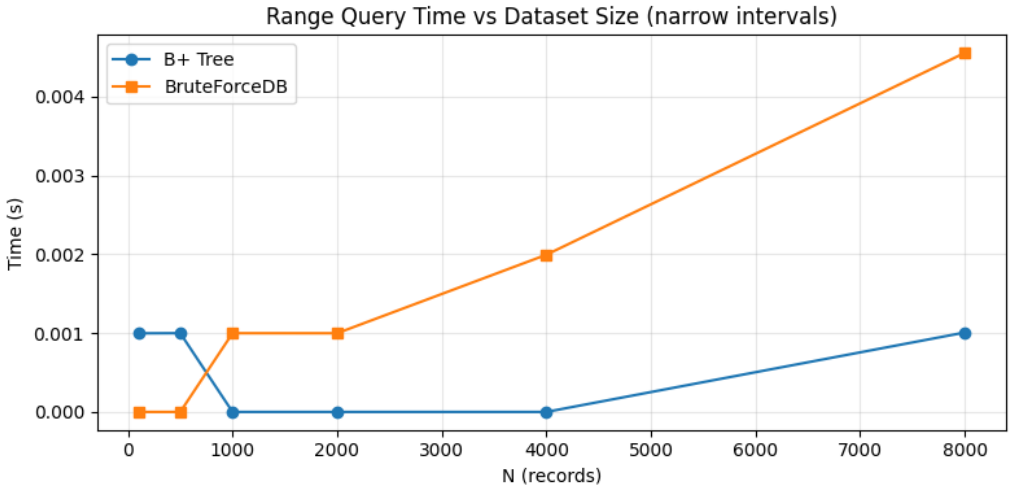

Figure 6: Range query time: B+ Tree vs BruteForceDB.

Analysis: With narrow query windows (width 100 to 1 000), the B+ Tree locates the start in $O(\log N)$ then walks only the K matching leaves. BruteForceDB must scan all N elements every time, causing its time to grow proportionally to N.

Deletion Time

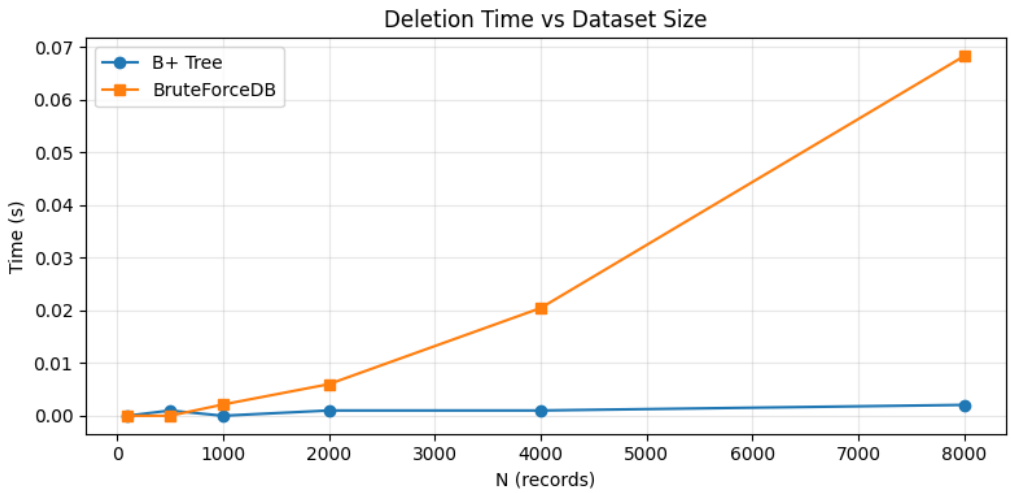

Figure 7: Deletion time: B+ Tree vs BruteForceDB.

Analysis: BruteForceDB must first locate the element ($O(N)$ scan) and then shift elements upon removal. The B+ Tree deletion, even accounting for borrowing and merging, remains $O(\log N)$, making it dramatically faster at scale.

Memory Usage (Peak at Insertion)

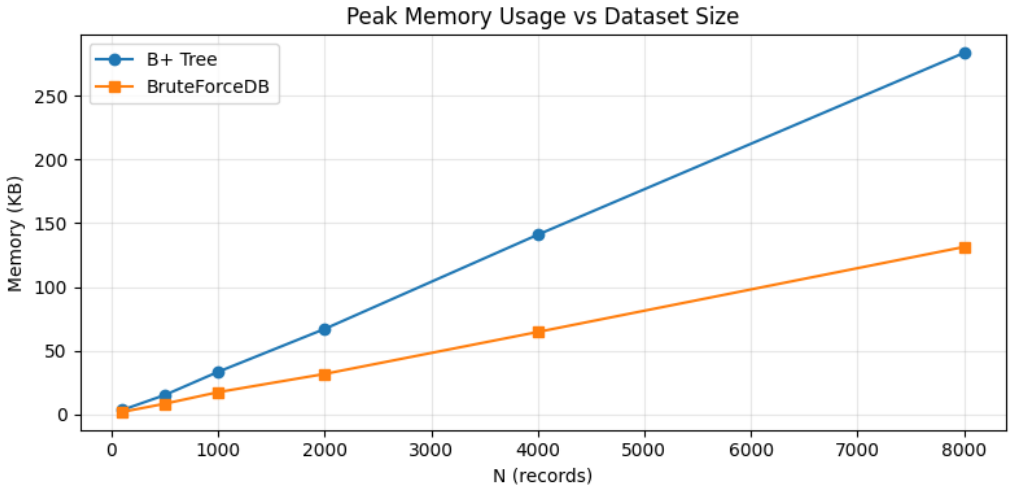

Figure 8: Peak memory usage during insertion: B+ Tree vs BruteForceDB.

Analysis: The B+ Tree consumes more memory per element due to node objects and pointers. BruteForceDB's raw list is more memory-compact. In real databases this trade-off is worthwhile because indexed lookups eliminate expensive full scans.

Comprehensive Tests

The Jupyter notebook includes systematic tests for all components:

1. **BPlusTree**: insertion, search, range query, delete, update, edge cases (duplicate keys, empty tree, single-element operations).
2. **BruteForceDB**: insert, search, delete, range query on the baseline structure.
3. **Table**: schema validation, record CRUD via the Table wrapper, range queries on both the search key and non-key fields.
4. **DatabaseManager**: database and table creation/deletion, listing, cross-database isolation.
5. **Flask Endpoints**: end-to-end HTTP tests using Flask's test client for all REST API routes (CRUD, search, range, visualise).

Module A: Conclusion

Metric	Winner	Reason
Insertion	BruteForceDB (small N) / B+ Tree (large N)	List append is $O(1)$; B+ Tree splits add constant overhead per insert.
Search	**B+ Tree**	$O(\log N)$ vs $O(N)$.
Range Query	**B+ Tree**	Leaf chain avoids full scan.
Deletion	**B+ Tree**	$O(\log N)$ vs $O(N)$ locate + shift.
Memory	BruteForceDB	Plain list has lower per-element overhead.

Table 6: Module A performance summary.

For any workload dominated by searches, range queries, or deletions, the B+ Tree is the clear winner. The BruteForceDB only excels at pure append-heavy workloads with no retrieval, a scenario rarely encountered in real databases. The tree visualisations confirm correct node splitting, merging, and leaf-chain maintenance.

Video Link: `INSERT-MODULE-A-VIDEO-URL-HERE`

Module B: Local API Development, RBAC & Database Optimisation

Local Database & Secure API Implementation

Tech Stack

Layer	Technology
Frontend	React 19, Vite 8, React Router 7
Backend	Flask, Flask-JWT-Extended, Flask-CORS
Database	MySQL 8.0+ (raw SQL, no ORM)
Auth	JWT (JSON Web Tokens), bcrypt, Email OTP
Email	Gmail SMTP via `python-dotenv`

Table 7: Technology stack used in IITGN Connect.

The entire backend uses **raw SQL queries**, no ORM (SQLAlchemy, Django ORM, etc.) is used anywhere in the project. All database interaction goes through three helper functions in `db.py`: `query_db()` for SELECT queries, `execute_db()` for single INSERT/UPDATE/DELETE statements, and `execute_transaction()` for multi-statement atomic operations.

Database Schema (22 Tables)

The schema follows an **ISA (Generalization/Specialization) hierarchy** for member types. The `Member` table is the supertype containing shared attributes (credentials, contact info, privacy settings), while `Student`, `Professor`, `Alumni`, and `Organization` are subtypes storing type-specific attributes. Each subtype's primary key is also a foreign key referencing `Member(MemberID)` with `ON DELETE CASCADE`.

Category	Tables
Core (ISA)	`Member` (supertype), `Student`, `Professor`, `Alumni`, `Organization`
Content	`Post`, `Comment`, `PostLike`, `Poll`, `PollOption`, `PollVote`
Groups	`CampusGroup`, `GroupMembership`
Academic	`Course`, `Enrollment`, `ClassAttendance`, `MessAttendance`
Jobs	`JobPost`, `ReferralRequest`
Profile	`ProfileClaimQuestion`, `ProfileClaimVote`
Security	`AuditLog`, `OTPVerification`

Table 8: Database tables organized by functional category.

7.2.1 Data Integrity Strategy

- **ISA hierarchy via shared PK/FK**: Subtype tables use `MemberID` as both their primary key and a foreign key to `Member`, enforcing a strict 1:1 relationship with

cascading deletes.

- **No credential duplication**: Login credentials (username, password hash, email) are stored only in the `Member` table. Subtype tables never duplicate this data.
- **Cascade deletes on all FK relationships**: Deleting a `Member` automatically removes all their posts, comments, likes, poll votes, group memberships, job posts, referral requests, and profile claims.
- **Composite primary keys for join tables**: `PostLike(PostID, MemberID)`, `GroupMembership(MemberID)`, `PollVote(OptionID, MemberID)`, `Enrollment(StudentID, CourseID, Semester)`. These prevent duplicate entries at the database level.
- **ENUM constraints**: `MemberType`, `Status`, `MealType`, `Role` columns use MySQL `ENUM` types to enforce valid values.

7.2.2 Key Table Definitions

```sql
CREATE TABLE Member (
    MemberID INT AUTO_INCREMENT PRIMARY KEY,
    Username VARCHAR(50) UNIQUE NOT NULL,
    Name VARCHAR(100) NOT NULL,
    Email VARCHAR(100) UNIQUE NOT NULL,
    Password VARCHAR(256) NOT NULL,
    MemberType ENUM('Student','Professor','Alumni','Organization') NOT NULL,
    ContactNumber VARCHAR(15),
    CreatedAt DATE NOT NULL,
    AvatarColor VARCHAR(10) DEFAULT '#4F46E5',
    IsAdmin BOOLEAN DEFAULT FALSE,
    ShowEmail BOOLEAN DEFAULT TRUE,
    ShowContact BOOLEAN DEFAULT TRUE,
    AllowQnA BOOLEAN DEFAULT TRUE
);
```

Listing 7: Member table (supertype) with privacy controls

```sql
CREATE TABLE Student (
    MemberID INT PRIMARY KEY,
    Programme VARCHAR(20) NOT NULL,
    Branch VARCHAR(50) NOT NULL,
    CurrentYear INT NOT NULL,
    MessAssignment VARCHAR(20),
    FOREIGN KEY (MemberID) REFERENCES Member(MemberID) ON DELETE CASCADE
);
```

Listing 8: ISA subtype example: Student table

```sql
CREATE TABLE AuditLog (
    LogID INT AUTO_INCREMENT PRIMARY KEY,
    Timestamp DATETIME DEFAULT CURRENT_TIMESTAMP,
    Username VARCHAR(100),
    Action VARCHAR(50),
    Endpoint VARCHAR(200),
    IPAddress VARCHAR(50),
    Details TEXT,
    IsAuthorized BOOLEAN DEFAULT TRUE
);
```

Listing 9: AuditLog table for security tracking

CRUD API Endpoints

The backend exposes **56 RESTful API endpoints** across 10 route blueprints (`auth.py`, `posts.py`, `groups.py`, `jobs.py`, `polls.py`, `attendance.py`, `profile.py`, `members.py`, `admin.py`, `settings.py`). Every data-modifying endpoint uses parameterized SQL queries to prevent SQL injection.

Method	Endpoint	Description	Auth
POST	`/api/auth/login`	Login (username or email)	Public
POST	`/api/auth/register`	Register new account	Public
POST	`/api/auth/send-otp`	Send OTP to email	Public
POST	`/api/auth/verify-otp`	Verify OTP code	Public
POST	`/api/auth/forgot-password`	Request password reset OTP	Public
POST	`/api/auth/reset-password`	Reset password with OTP	Public
GET	`/api/auth/isAuth`	Check token validity	JWT
GET	`/api/posts`	Get feed (global or groups)	JWT
POST	`/api/posts`	Create post (image upload)	JWT
PUT	`/api/posts/:id`	Update post (author only)	JWT
DELETE	`/api/posts/:id`	Delete post (author/admin)	JWT
POST	`/api/posts/:id/like`	Toggle like	JWT
GET	`/api/posts/:id/comments`	Get comments	JWT
POST	`/api/posts/:id/comments`	Add comment	JWT
PUT	`/api/comments/:id`	Edit comment (author only)	JWT
DELETE	`/api/comments/:id`	Delete comment (author/admin)	JWT
GET	`/api/groups/`	List all groups	JWT
POST	`/api/groups/`	Create group	JWT
PUT	`/api/groups/:id`	Update group (admin only)	JWT
DELETE	`/api/groups/:id`	Delete group (admin only)	JWT
POST	`/api/groups/:id/join`	Join group	JWT
POST	`/api/groups/:id/leave`	Leave group	JWT
GET	`/api/groups/:id/pending`	Pending join requests	JWT
POST	`/api/groups/:id/approve/:mid`	Approve member (group admin)	JWT
POST	`/api/groups/:id/reject/:mid`	Reject member (group admin)	JWT
POST	`/api/groups/:id/kick/:mid`	Kick member (group admin)	JWT
GET	`/api/polls/`	List polls	JWT
POST	`/api/polls/`	Create poll	JWT
POST	`/api/polls/:id/vote`	Vote on poll	JWT
GET	`/api/jobs`	List job postings	JWT
POST	`/api/jobs`	Post job (alumni only)	JWT
DELETE	`/api/jobs/:id`	Delete job (author/admin)	JWT
GET	`/api/referrals`	Get referral requests	JWT
POST	`/api/referrals`	Create referral request	JWT

Method	Endpoint	Description	Auth
PUT	`/api/referrals/:id`	Update referral status	JWT
GET	`/api/attendance/class`	Class attendance records	JWT
GET	`/api/attendance/mess`	Mess attendance records	JWT
GET	`/api/attendance/streaks`	Attendance streaks	JWT
GET	`/api/attendance/leaderboard`	Streak leaderboard	JWT
GET	`/api/profile/:id`	Get user profile	JWT
GET	`/api/settings/privacy`	Get privacy settings	JWT
PUT	`/api/settings/privacy`	Update privacy settings	JWT
PUT	`/api/settings/profile`	Update profile	JWT
PUT	`/api/settings/password`	Change password	JWT
DELETE	`/api/settings/account`	Delete account	JWT
GET	`/api/admin/stats`	System statistics	Admin
GET	`/api/admin/members`	List all members	Admin
PUT	`/api/admin/members/:id`	Update member role	Admin
DELETE	`/api/admin/members/:id`	Delete member	Admin
POST	`/api/admin/query`	Execute raw SQL	Admin

Table 9: Complete API endpoint listing with authentication requirements.

Session Validation

Authentication uses **JWT (JSON Web Tokens)** via the Flask-JWT-Extended library:

- **Token issuance**: On successful login (`POST /api/auth/login`), the server creates a JWT with `identity = MemberID` and 24-hour expiry.
- **Token validation**: Every protected endpoint uses the `@jwt_required()` decorator, which automatically validates the token signature, checks expiry, and extracts the user identity.
- **Session check**: `GET /api/auth/isAuth` allows the frontend to verify a stored token is still valid and retrieve the user's role/type.
- **Password security**: All passwords are hashed with `bcrypt` (with per-user salt). No plaintext passwords are stored or transmitted in responses.

```python
@auth_bp.route('/login', methods=['POST'])
def login():
    data = request.get_json()
    login_id = data.get('username', '').strip()
    password = data.get('password', '')

    member = query_db(
        "SELECT * FROM Member WHERE Username = %s OR Email = %s",
        (login_id, login_id), one=True
    )
    if not member or not bcrypt.checkpw(
        password.encode(), member['Password'].encode()
    ):
        return jsonify(error='Invalid credentials'), 401

    token = create_access_token(
```

```
        identity=str(member['MemberID']),
        expires_delta=timedelta(hours=24)
    )
    return jsonify(token=token, user={...})
```

Listing 10: JWT-based session validation flow in auth.py

Member Portfolio (Profile)

Each member has a profile page (`GET /api/profile/:id`) that aggregates data from multiple tables:

- Personal information from `Member` + subtype-specific details (Student: programme/branch/year; Professor: designation/department; Alumni: organization/graduation year)
- Recent posts with like and comment counts (via correlated subqueries)
- Group memberships with roles
- Profile Claims (Q&A) with community agree/disagree voting
- **Privacy controls**: Members can toggle visibility of email, contact number, and Q&A section via `PUT /api/settings/privacy`. The backend enforces these settings, hidden fields are returned as `null` in the API response, not just hidden in the UI.

Security & Role-Based Access Control (RBAC)

Role-Based Access Control

The platform enforces access control at multiple levels:

Role	Permissions
Admin	Full CRUD on all members, groups, posts, comments, jobs. System stats dashboard. Raw SQL console. Self-delete protection.
Student	Create/edit/delete own content. Submit referral requests. View attendance.
Alumni	Regular permissions + create/edit/delete job postings. Manage referral requests (approve/reject).
Professor	Regular permissions.
Organization	Regular permissions.

Table 10: RBAC permission matrix by member type.

8.1.1 Implementation Details

Admin enforcement uses a decorator pattern in `admin.py`:

```
@admin_bp.before_request
@jwt_required()
def check_admin():
    user_id = int(get_jwt_identity())
    member = query_db(
        "SELECT IsAdmin, Username FROM Member WHERE MemberID = %s",
        (user_id,), one=True
    )
    if not member or not member['IsAdmin']:
        log_action('FORBIDDEN_ACCESS',
            f"Non-admin user '{member['Username']}' attempted "
            f"{request.method} {request.path}",
            user=member['Username'] if member else f'uid:{user_id}')
        return jsonify(error='Admin access required'), 403
```

Listing 11: Admin access check with RBAC violation logging

Ownership checks are enforced on edit/delete operations. For example, editing a post:

```
post = query_db("SELECT AuthorID FROM Post WHERE PostID = %s",
                (post_id,), one=True)
if post['AuthorID'] != user_id and not is_admin:
    return jsonify(error='Unauthorized'), 403
```

Listing 12: Ownership verification before edit

Type-based restrictions: Job posting is restricted to `MemberType = 'Alumni'`. Referral requests are restricted to Students. These checks happen in the route handlers before any database modification.

Role-Based UI Visibility

The frontend conditionally renders sidebar items and page sections based on the user's role:

Feature	Student	Alumni	Prof.	Org.	Admin
Jobs & Referrals	View+Refer	Post Jobs	Hidden	Hidden	View+Delete
My Referrals Tab	Yes	Requests	N/A	N/A	Hidden
Attendance	Full	Hidden	Hidden	Full	Leaderboard
Admin Dashboard	Hidden	Hidden	Hidden	Hidden	Full
SQL Console	Hidden	Hidden	Hidden	Hidden	Full

Table 11: Feature visibility by member type in the UI.

Proper Deletion Integrity

All foreign key relationships use `ON DELETE CASCADE`, ensuring referential integrity is maintained automatically:

- Deleting a **Member** removes their subtype row, all posts, comments, likes, poll votes, group memberships, job posts, referral requests, profile claims, and claim votes.
- Deleting a **Post** removes all its comments and likes.
- Deleting a **Poll** removes all its options and votes.
- Deleting a **CampusGroup** removes all memberships.
- All admin delete operations are logged in the audit trail.

Unauthorized Access Logging

The platform uses a **two-layer security logging system** to detect and record unauthorized access:

8.4.1 Layer 1: Unauthorized API Request Detection

Custom JWT error handlers in `app.py` catch and log all authentication failures to `logs/audit.log`:

```
@jwt.unauthorized_loader
def unauthorized_callback(reason):
    log_action('UNAUTHORIZED_ACCESS',
        f'Missing/invalid JWT on {request.method} {request.path}',
        user='UNAUTHORIZED')
    return jsonify(error='Missing or invalid token'), 401

@jwt.expired_token_loader
def expired_callback(jwt_header, jwt_payload):
    user_id = jwt_payload.get('sub', 'unknown')
    log_action('UNAUTHORIZED_ACCESS',
        f'Expired JWT (user_id={user_id}) on '
        f'{request.method} {request.path}',
        user='EXPIRED_TOKEN')
    return jsonify(error='Token has expired'), 401

```

```python
@jwt.invalid_token_loader
def invalid_callback(reason):
    log_action('UNAUTHORIZED_ACCESS',
        f'Invalid/tampered JWT on {request.method} {request.path}',
        user='INVALID_TOKEN')
    return jsonify(error='Invalid token'), 401
```

Listing 13: JWT error handlers for unauthorized access logging

Example log entries:

```
[USER:UNAUTHORIZED] [ACTION:UNAUTHORIZED_ACCESS] -- Missing JWT on GET /api/posts
[USER:EXPIRED_TOKEN] [ACTION:UNAUTHORIZED_ACCESS] -- Expired JWT on GET /api/posts
[USER:laksh_jain] [ACTION:FORBIDDEN_ACCESS] -- Non-admin attempted GET /api/admin/stats
```

8.4.2 Layer 2: Direct DB Modification Detection (63 MySQL Triggers)

63 MySQL triggers (INSERT/UPDATE/DELETE on all 21 data tables) detect operations that bypass the API entirely:

- **API operations**: Before every SQL query, `db.py` sets the MySQL session variable `@app_username` to the authenticated user's username. Triggers read this variable and log with `IsAuthorized = TRUE`.
- **Direct SQL access**: When someone runs SQL directly in the MySQL CLI (bypassing the API), `@app_username` is `NULL`. Triggers detect this and log with `IsAuthorized = FALSE` and `Username = 'DIRECT_DB_ACCESS'`.

```python
def _set_audit_session_vars(cursor):
    """Set MySQL session vars so triggers identify API operations."""
    if has_request_context():
        username = get_current_username()  # from JWT
        cursor.execute("SET @app_username = %s", (username,))
        cursor.execute("SET @app_endpoint = %s", (request.path,))
        cursor.execute("SET @app_ip = %s", (request.remote_addr,))
```

Listing 14: Setting session variables before every query (db.py)

```cpp
CREATE TRIGGER trg_post_insert AFTER INSERT ON Post
FOR EACH ROW
BEGIN
    INSERT INTO AuditLog (Username, Action, Endpoint, IPAddress,
                          Details, IsAuthorized)
    VALUES (
        COALESCE(@app_username, 'DIRECT_DB_ACCESS'),
        'INSERT_POST',
        COALESCE(@app_endpoint, 'DIRECT_SQL'),
        COALESCE(@app_ip, 'N/A'),
        CONCAT('PostID=', NEW.PostID, ', AuthorID=', NEW.AuthorID),
        CASE WHEN @app_username IS NOT NULL THEN TRUE ELSE FALSE END
    );
END;
```

Listing 15: Example trigger for detecting direct DB modifications

To verify:

```sql
-- Run directly in MySQL CLI (bypasses API)
UPDATE Post SET Content = 'Hacked!' WHERE PostID = 1;

-- Check AuditLog -- flagged as unauthorized
SELECT * FROM AuditLog WHERE IsAuthorized = FALSE AND Action = 'UPDATE_POST';
```

Database Optimization

Indexing Strategy

We analyzed every SQL query across all 10 Flask route handler files and identified columns used in `WHERE`, `JOIN`, `ORDER BY`, and correlated subqueries. Based on this analysis, we created **26 custom indexes** (defined in `sql/indexes.sql`).

9.1.1 Index Design Principles

1. **Single-column indexes** for simple WHERE/JOIN lookups (e.g., `idx_post_authorid` on `Post.AuthorID`).
2. **Composite indexes** that cover both filter and sort columns (e.g., `idx_post_groupid_createdat` covers `WHERE GroupID = ? ORDER BY CreatedAt DESC`).
3. **Descending key parts** using MySQL 8.0+ DESC index support for `ORDER BY ... DESC` optimization.
4. **Covering indexes** for correlated subqueries that only need COUNT operations.

9.1.2 Complete Index Listing

#	Table	Index	Column(s)	Query Pattern
1	Post	`idx_post_authorid`	AuthorID	Profile: posts by author
2	Post	`idx_post_groupid`	GroupID	Group feed filter
3	Post	`idx_post_groupid_createdat`	(GroupID, CreatedAt↓)	Group posts sorted by date
4	Post	`idx_post_createdat`	CreatedAt↓	Global feed ORDER BY
5	Post	`idx_post_authorid_createdat`	(AuthorID, CreatedAt↓)	Profile: recent posts
6	Comment	`idx_comment_postid_createdat`	(PostID, CreatedAt↑)	Comments sorted
7	Comment	`idx_comment_postid`	PostID	Comment count subquery
8	Comment	`idx_comment_authorid`	AuthorID	Comment author JOIN
9	PostLike	`idx_postlike_memberid`	MemberID	Liked posts by user
10	GroupMbr	`idx_grpmembership_memberid`	MemberID	User's groups lookup
11	CampusGroup	`idx_campusgroup_adminid`	AdminID	Group admin JOIN
12	Member	`idx_member_name`	Name	Member search ORDER BY
13	Member	`idx_member_membertype`	MemberType	Member type filter
14	Poll	`idx_poll_createdat`	CreatedAt↓	Poll listing sort
15	Poll	`idx_poll_creatorid`	CreatorID	Poll creator JOIN
16	PollOption	`idx_polloption_pollid`	PollID	Options per poll
17	PollVote	`idx_pollvote_memberid`	MemberID	User vote check
18	JobPost	`idx_jobpost_postedat`	PostedAt↓	Job listing sort
19	JobPost	`idx_jobpost_alumniid`	AlumniID	Job poster JOIN
20	ReferralReq	`idx_ref_targetalumniid`	(TargetAlumniID, ReqAt↓)	Alumni referral dash.
21	ReferralReq	`idx_ref_studentid`	(StudentID, ReqAt↓)	Student referral hist.
22	ClassAtt	`idx_classatt_student_date`	(StudentID, RecordDate)	Attendance by student
23	ClassAtt	`idx_classatt_courseid`	CourseID	Course JOIN
24	MessAtt	`idx_messatt_student_date`	(StudentID, RecordDate)	Mess attendance lookup
25	ProfileClaim	`idx_profileclaim_memberid`	MemberID	Claims per profile
26	Enrollment	`idx_enrollment_courseid`	CourseID	Course detail lookup

Table 12: All 26 custom performance indexes with rationale.

Query Profiling

9.2.1 Benchmark Methodology

The benchmarking script (`app/backend/benchmark.py`) measures performance across **10 representative queries** covering the most critical API endpoints:

1. Global Post Feed: full feed with author info, like/comment counts
2. Group Posts: group-filtered feed with counts
3. Post Comments: comments per post with author details
4. User Profile Posts: recent posts by a specific member
5. Class Attendance: monthly attendance with course info
6. Mess Attendance: monthly mess records
7. Job Listings: all jobs with alumni info
8. Referral Requests: alumni's incoming referrals
9. Polls with Options & Votes: poll listing with creator info
10. Member Search by Type: filtered member directory

Methodology:

- **Bulk data generation**: ∼2,000 posts, 5,000 comments, 8,000 likes, 10,000+ attendance records, 230 members to ensure the optimizer considers indexes worthwhile.
- **IGNORE INDEX hints**: "Before" queries use `IGNORE INDEX` hints to force the MySQL optimizer to skip all non-primary indexes, simulating a no-index state without actually dropping indexes.
- **Normal queries**: "After" queries run without hints, using all available indexes.
- **100 iterations** per query for statistical reliability.
- **EXPLAIN analysis** on both versions to verify access type changes.

9.2.2 Why IGNORE INDEX Instead of DROP/CREATE?

MySQL InnoDB automatically creates indexes for foreign key columns. These auto-generated FK indexes **cannot be dropped** even with `SET FOREIGN_KEY_CHECKS = 0`. This means a naive "drop all indexes, benchmark, re-create" approach would still show FK indexes in the "before" EXPLAIN plans, making them identical to the "after" plans for many queries.

Using `IGNORE INDEX` hints allows us to force the optimizer to ignore specific indexes on a per-query basis, producing genuine "no-index" behavior for benchmarking while keeping the database in a consistent state.

Optimization Report

Execution Time Results

We report two timing metrics: **Exec Time** is the wall-clock average over 100 runs (includes Python driver overhead), and **EA Time** is the actual execution time reported by MySQL's `EXPLAIN ANALYZE` (purely server-side, no driver overhead).

Query	Exec Before (ms)	Exec After (ms)	EA Before (ms)	EA After (ms)	Speedup
Global Post Feed	3444.82	14.03	67.70	4.91	**99.6%**
Group Posts	679.74	3.16	16.80	2.23	**99.5%**
Post Comments	1.83	0.20	1.71	0.008	**89.0%**
User Profile Posts	11.19	0.28	0.68	0.023	**97.5%**
Class Attendance	3.25	0.80	9.08	0.54	**75.3%**
Mess Attendance	2.64	0.97	3.24	0.94	**63.4%**
Job Listings	0.71	0.85	0.41	0.28	−19.8%
Referral Requests	0.28	0.20	0.18	0.021	**29.6%**
Polls with Options	0.67	0.67	0.20	0.16	−1.0%
Member Search by Type	0.76	0.91	0.20	0.27	−20.0%
Average					**∼51%**

Table 13: Benchmark results: *Exec* = wall-clock avg (100 runs); *EA* = MySQL EXPLAIN ANALYZE actual time.

Key observations:

- The top 4 queries (Global Feed, Group Posts, Profile Posts, Post Comments) show **89–99%+ speedup** because they involve correlated subqueries scanning the Comment and PostLike tables for every row in the Post table. Without indexes, each subquery does a full table scan (∼5,000 rows for Comments, ∼8,000 for PostLike). With indexes, these become covering index lookups returning 2–4 rows each.
- Queries on small tables (Jobs, Polls, Members) show **negligible or slightly negative speedup** because the tables are small enough that full table scans are already fast, and the overhead of index maintenance slightly outweighs the benefit.
- The **Global Post Feed** improved from 3.4 seconds to 14 milliseconds, a **245**× speedup, because the dependent subqueries went from scanning 5,000+ rows each to looking up 2–4 rows via covering index lookups.
- **EXPLAIN ANALYZE times** closely track wall-clock times for large queries but diverge for small ones (due to Python driver/network overhead in wall-clock measurements).

Scan Type Analysis (EXPLAIN ANALYZE)

The benchmark uses MySQL 8.0+ `EXPLAIN ANALYZE` to capture the **actual scan types** used by the optimizer at runtime, not just the estimated plan.

Query	Scan Types Before	Scan Types After
Global Post Feed	Full Table Scan, Index Lookup, Index Lookup	Full Table Scan, Covering Index Lookup (x2)
Group Posts	Full Table Scan, Full Index Scan, Index Lookup	Index Lookup, Covering Index Lookup (x2)
Post Comments	Full Table Scan	Index Lookup
User Profile Posts	Full Index Scan (x2), Index Lookup	Index Lookup, Covering Index Lookup (x2)
Class Attendance	Full Table Scan (x2)	Index Lookup
Mess Attendance	Full Table Scan	Index Lookup
Job Listings	Full Table Scan, Index Lookup	Full Table Scan, Index Lookup
Referral Requests	Full Table Scan, Index Lookup	Index Lookup
Polls	Full Table Scan	Full Table Scan
Member Search	Full Table Scan	Index Lookup

Table 14: Scan type comparison from EXPLAIN ANALYZE. "Before" queries use IGNORE INDEX hints; "After" queries use all available indexes.

Scan type definitions:

- **Full Table Scan**: reads every row in the table (worst case, $O(n)$).
- **Full Index Scan**: reads every entry in an index (better than table scan but still $O(n)$).
- **Index Lookup**: uses an index to directly locate matching rows ($O(\log n)$).
- **Covering Index Lookup**: resolves the query entirely from the index without touching the data pages (best case for COUNT subqueries).

Benchmarking Charts

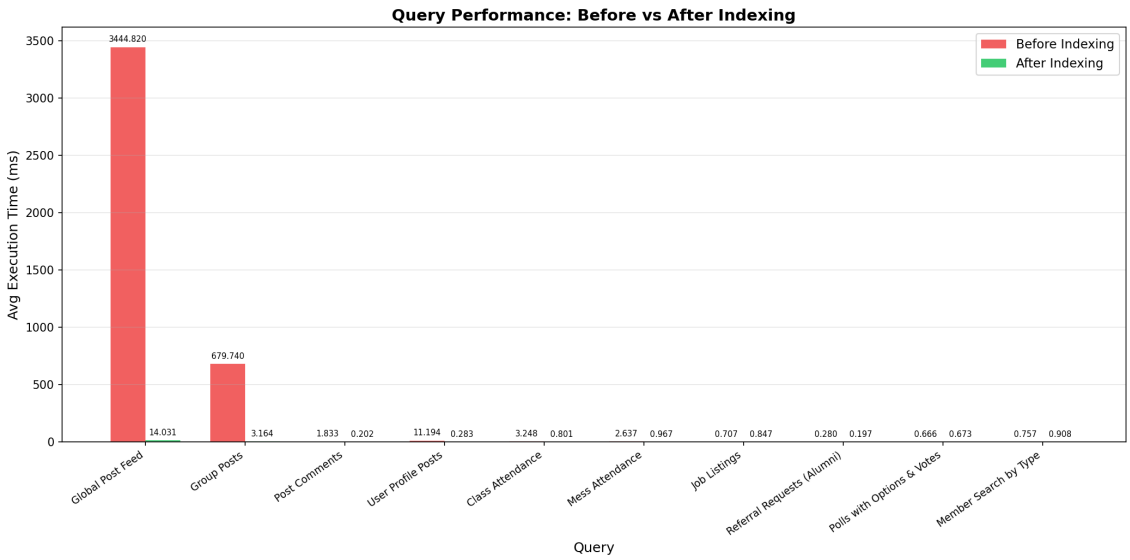

Figure 9: Execution time comparison: before (no indexes) vs. after (with indexes) for each benchmark query.

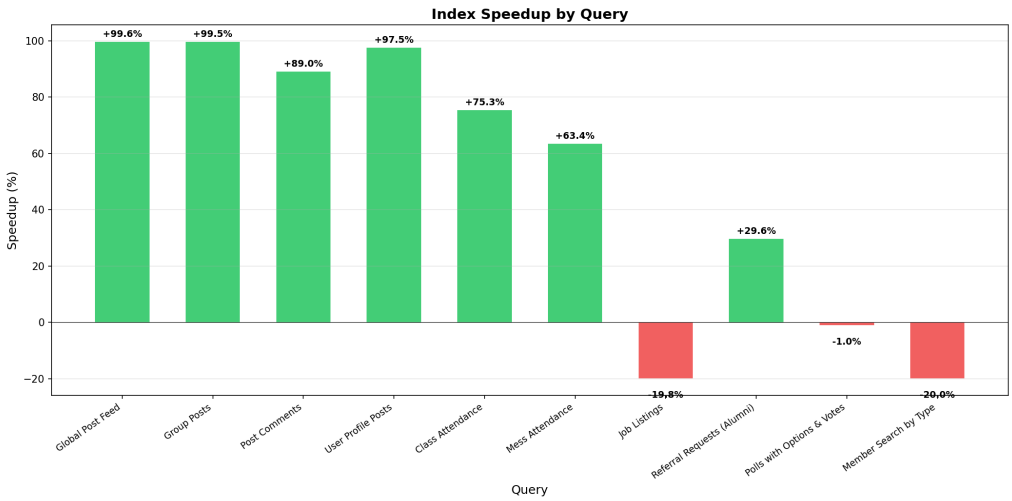

Figure 10: Speedup percentage per query. Positive values indicate improvement; negative values indicate slight overhead on small tables.

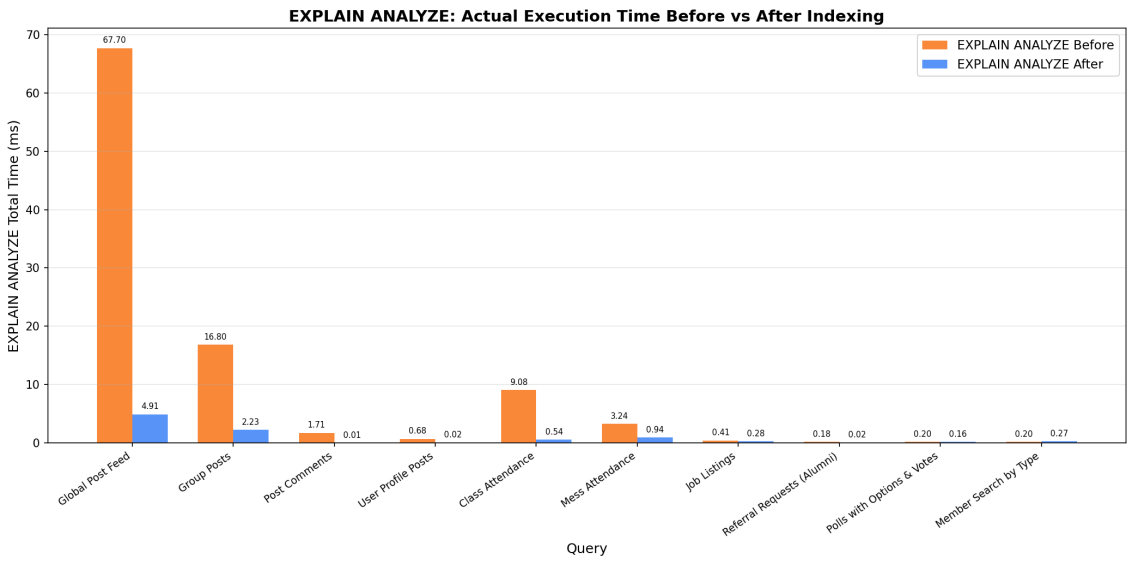

Figure 11: EXPLAIN ANALYZE actual execution times: server-side timing before vs. after indexing (no driver overhead).

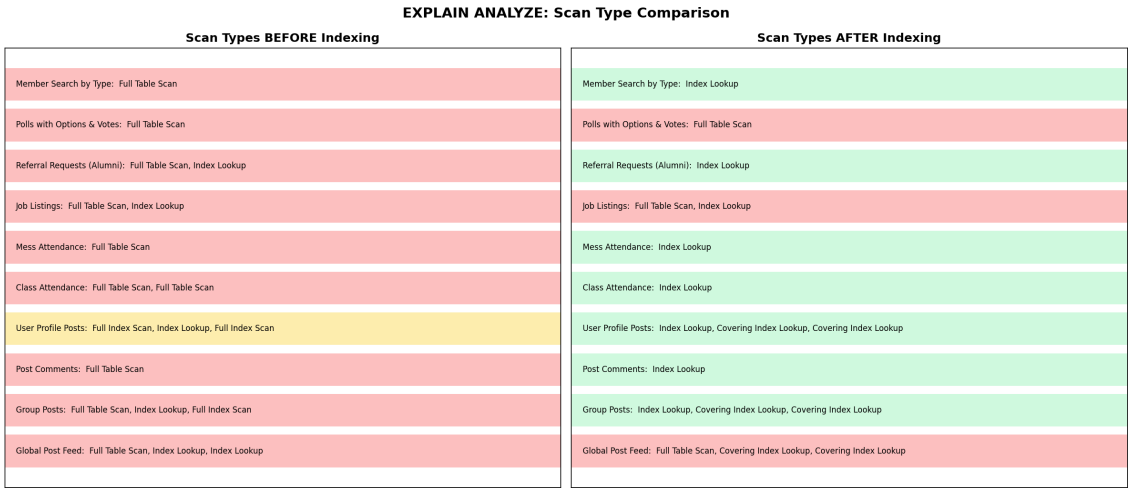

Figure 12: EXPLAIN ANALYZE scan type comparison. Red = full table scans; yellow = full index scans; green = index lookups.

EXPLAIN Plan Analysis

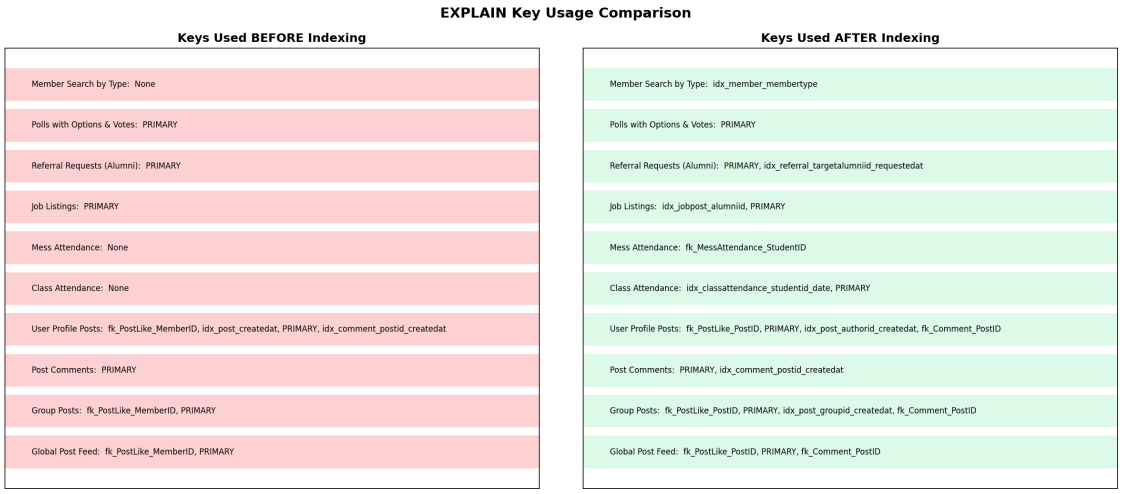

Figure 13: EXPLAIN key usage comparison showing which indexes are used before vs. after for each query.

10.4.1 Detailed EXPLAIN Comparison: Global Post Feed

Table	Type Before	Type After	Key Before	Key After	Rows
Post (p)	ALL	ALL	None	None (full scan)	2017
Member (m)	eq_ref	eq_ref	PRIMARY	PRIMARY	1
CampusGroup	eq_ref	eq_ref	PRIMARY	PRIMARY	1
Comment	**ALL**	**ref**	**None**	**fk_Comment_PostID**	**2**
PostLike	**index**	**ref**	**fk_PostLike_MID (7957)**	**fk_PostLike_PostID**	**4**

Table 15: EXPLAIN comparison for Global Post Feed. Comment subquery drops from 4,996-row full scan to 2-row index lookup; PostLike drops from 7,957 rows to 4 rows.

10.4.2 Detailed EXPLAIN Comparison: Group Posts

Table	Type Before	Type After	Key Before	Key After	Rows
Post (p)	ALL (2017)	ref	None	idx_post_groupid_createdat	336
Comment	ALL (4996)	ref	None	fk_Comment_PostID	2
PostLike	index (7957)	ref	fk_PostLike_MemberID	fk_PostLike_PostID	4

Table 16: EXPLAIN comparison for Group Posts. Post table scan narrows from 2,017 to 336 rows; subqueries use direct lookups.

Video Walkthrough

Video Link: https://www.youtube.com/watch?v=QTxnMOmavAc

The video demonstration covers the following areas:

UI & API Functionality

Complete walkthrough of all platform features:

- User registration with email OTP verification
- Login/logout with JWT session management
- Global feed: creating, editing, deleting posts with image uploads
- Group management: creating, joining, leaving, admin approval for restricted groups
- Polls: creating and voting
- Job board: alumni posting jobs, students requesting referrals
- Attendance tracking: class and mess attendance, calendar view, streaks
- User profiles: portfolio view, profile claims with voting
- Settings: profile editing, password change, privacy toggles
- Member directory with search and type filtering

RBAC Constraints

Demonstration of role-based access:

- Admin login showing dashboard, member management, SQL console
- Regular user login showing restricted sidebar (no admin features)
- Attempting to access admin endpoints as regular user: 403 response + audit log entry
- Alumni-only job posting, student-only referral requests
- Ownership enforcement: cannot edit/delete other users' content

Security Logging

Live demonstration of the audit system:

- Unauthorized API access: calling protected endpoint without JWT token
- Expired token handling: using an expired JWT
- RBAC violation: non-admin user hitting `/api/admin/stats`
- Direct DB modification detection: running an UPDATE directly in MySQL CLI and checking the AuditLog table for `IsAuthorized = FALSE`

Team Member Contributions

Member	Contributions
Laksh Jain	Module A: B+ Tree implementation, performance benchmarking, tree visualisation, Flask REST API
Rudra Pratap Singh	Module A: B+ Tree implementation, Table/DatabaseManager abstractions, testing, report
Parthiv Patel	Module B: Database schema design, backend API development (routes, db helpers), seed data generation, privacy settings implementation
Shriniket Behera	Module B: RBAC enforcement, indexing strategy, benchmarking script, optimization report
Ridham Patel	Module B: Frontend development (React pages, components, routing), UI/UX design, API integration, security implementation (JWT handlers, audit logging, MySQL triggers)